

C Programming Book

Mohan Arunachalam

Contents

1	Introduction to C	3
1.1	List of all Operators	4
1.2	Assignment Operator	5
1.3	Arithmetic Operators	5
1.4	C-Tokens	7
1.5	printf function	7
1.6	Variable definition Rules	9
1.7	Relational Operators	12
1.8	Logical Operator	13
2	Bitwise Operators	15
2.1	One's Complement	15
2.1.1	Binary(2's complement) to decimal	15
2.1.2	Summary	16
2.2	Left shift Operator	17
2.2.1	Summary	17
2.3	Right shift Operator	17
2.3.1	Summary	17
2.4	Bitwise AND, OR, XOR	18
2.4.1	Summary	18
2.5	Problem solving	18
2.5.1	Check if the nth bit is set	18
2.5.2	Set the nth bit	18
2.5.3	Clear the nth bit	19
2.5.4	Toggle the nth bit	19
2.5.5	Count number of set bits	19
2.5.6	Check if a number is a power of 2	19
2.5.7	Swap two numbers using XOR	19
3	RTOS	20
3.1	Fundamentals	20
3.2	Concepts and Recommendations	20
3.2.1	The Pitfalls of Dynamic Memory (malloc)	20
3.2.2	Task States and Timing Analysis	20
3.2.3	Queues: The Right Way to Pass Data	20
3.2.4	Software Timers vs. Hardware Timers	21
3.2.5	The Tick Hook & System Heartbeat	21
3.2.6	The Idle Task & Power Management	21
3.2.7	Event Groups (The "Wait for All" logic)	21
3.3	Problem solving	21
3.3.1	Odd / Even Print	21
3.3.2	The Producer-Consumer Pattern	22
3.3.3	Queue Overflow	23
3.3.4	Handling Shared Hardware (The Re-entrancy Problem)	24
3.3.5	The Deadlock Scenario	25
3.3.6	Priority Inversion:	25

3.3.7	Interrupt to Task Communication	26
-------	---	----

Chapter 1

Introduction to C

- Comparison between English language and C programming language

English Language	C Language
1. Alphabets (a, b, c, ...)	1. Keywords (32)
2. Words (More than 10K)	2. Operators (45)
3. Sentence (Grammer makes this)	3. Separators (14)
	4. Constants

- Constants

Item	value	C datatype
Integer	5 , -5 , 0	int
real	5.12 , 0.39 , 5.00 , -3.25	float
charater	'a' , '5'	char

- Reason for the name float but why not real?
 - 100. 231452156
 - * . is decimal point
 - * 231452156 : This keep on changing So float
- Primary / Primitive / Fundamental / Basic data types in C
 - Data is a meaningfull information
 - Datatype is a different kind of data like int , float and Char

Format Specifier	Data Type	Description
%c	char	Prints a single character
%s	char *	Prints a string
%d	int	Signed decimal integer
%i	int	Signed integer (same as %d in printf)
%u	unsigned int	Unsigned decimal integer
%hd	short int	Signed short integer
%hu	unsigned short int	Unsigned short integer
%ld	long int	Signed long integer
%lu	unsigned long int	Unsigned long integer
%lld	long long int	Signed long long integer
%llu	unsigned long long int	Unsigned long long integer
%f	float	Floating-point number (printf promotes float to double)
%lf	double	Double-precision floating-point (scanf uses %lf for double)
%Lf	long double	Long double floating-point

Format Specifier	Data Type	Description
%e / %E	float, double	Scientific notation
%g / %G	float, double	Shortest representation of %f or %e
%x	unsigned int	Hexadecimal integer (lowercase)
%X	unsigned int	Hexadecimal integer (uppercase)
%o	unsigned int	Octal integer
%p	void *	Pointer address
%%	—	Prints % character
%zu	size_t	Unsigned size type
%td	ptrdiff_t	Pointer difference type
%jd	intmax_t	Largest signed integer type
%ju	uintmax_t	Largest unsigned integer type

1.1 List of all Operators

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
 	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
 	Logical OR	left to right
? :	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	comma operator	left to right

Figure 1.1: Operators

1.2 Assignment Operator

- It requires 2 arguments
- Left side arg must be variable
- Right side arg can be a variable / constant / expression
- Left side value is called l-value

```
1 // Not valid
2 a = 5
3 a = ;
4   = 10;
5 10 = 20;
6
7 // Valid
8 a = 5;
9 b = a;
10 c = 2 + 3;
```

- Every expression is replaced with constant

```
1 a = 5;
2 b = -3;
3 c = 9;
4
5 a = b; // a = -3
6 b = -c; // b = -9
7 a = -b; // a = 9
8
9 -c = a; // Error because l-value is expression not a variable
```

- In C language each and every statement ends with `;`
- Semicolon is called statement terminator (or) end of statement
- Why `.` is not used as terminator?
 - because `.` is used as decimal point in float number
- What is l-value error?
 - On the left hand side of assignment operation we should provide a variable but by mistake if we provide constant (or) expression then we get l-value required error.

1.3 Arithmetic Operators

```
1 // Example 1: Expression evaluation
2 a = 2 + 3;
3 a = 2 + 3 + 4;
4 a = 6 - 2 + 3 - 4 + 1;
5 a = 2 + 3 * 5;
6 a = (2 + 3) * 5;
7 a = 2 * 3 + 3 * 2;
```

Operation on	Result
int with int	int
int with float	float
float with int	float
float with float	float

```
1 // Example 2:
2 a = 5 / 2; // 2
3 a = -5 / 2; // -2
4 a = 5 / -2 // -2
```

```

5  a = -5 / -2 // -2
6
7  a = 5.0 / 2; // 2.5
8  a = 5 / 2.0; // 2.5
9  a = -5 / 2.0 // -2.5
10 a = 2 / 5 // 0
11 a = 2 / -5 // 0

```

```

1  // Example 3:
2
3  a = 17 / 3 / 2;
4  /*
5  a = 5 / 2
6  a = 2
7  */
8
9  a = 15 / 2 * 3;
10 /*
11 a = 7 * 3
12 a = 21
13 */
14
15 a = 9 * / 4;
16 /*
17 a = 27 / 4
18 a = 6
19 */
20
21 a = 6 * 3 / 4 * 5;
22 /*
23 a = 18 / 4 * 5
24 a = 4 * 5
25 a = 20
26 */

```

Division	Result of division	Modulus	Result of modulus
+ / +	+	+ % +	+
+ / -	-	+ % -	+
- / +	-	- % +	-
- / -	+	- % -	-

- Floating point numbers cannot used in modulus only integer is acceptable

```

1  // Example 1
2
3  a = 5 / -2; // -2
4  a = 5 % -2; // 1

```

```

1  // Example 2
2
3  a = 3.5 * 2 % 7; // Compilation error

```

- The result of modulus operation is remainder
- Result sign is same as numerator sign
 - If numerator is +ve then result is +ve
- Result sign does not depend on denominator
- Condition 1 : $x \% y == 0$ means then we can say that x is multiple of y
- Condition 2 : Numerator % 10 gives last digit of numerator

- Condition 3 : When numerator is smaller than denominator then result is numerator
- Condition 4 : When numerator is equal to denominator then result is zero

1.4 C-Tokens

- Smallest individual unit in C language is called C-Token
- C-Tokens are
 - Keywords
 - Operators
 - Separators
 - Constants
 - Identifiers
- There can be 'N' no. of space / Tabs / Newlines between two C-Tokens
- Program should be in good indentation

```
1 void main()
2 {
3     printf("Hello");
4 }
```

- The above program has following C-Tokens
 - Keyword: `void`
 - Operator: `(` `)` `(` `)`
 - Separator: `{` `;` `}`
 - Constant: `"Hello"`
 - Identifiers: `main` `printf`
- Total 11 C-Tokens are there in above program

1.5 printf function

- It will print 1st arg on screen
- 1st arg must be in pair of double quotes
- if more than one arg, then every arg must be separated by commas

```
1 printf("Hello");
2 // Hello
3
4 printf("    Hello    ");
5 // ...Hello...
6 // (Spaces are retained)
7
8 printf("Hello%dadc%d", 10, 20);
9 // Hello10abc20
10
11 printf("%d %d %d", 10, 20, 30);
12 // 10 20 30
13
14 printf("%d%d%d", 10, 20, 30);
15 // 102030
16
17 printf("%d,%d,%d", 10, 20, 30);
18 // 10,20,30
19
20 printf("%d %d %d", 10, 20);
21 // 10 20 <GV/Junk>
22
23 printf("%d %d %d", 10);
```



```

24 // 10 GV GV
25
26 printf("%d %d %d");
27 // GV GV GV
28
29 printf("%d %d %d", 10, 20, 30, 40);
30 // 10 20 30
31
32 printf("%d %f %c", 10, 3.75, 'a');
33 // 10 3.75 a
34
35 printf("%d", 5.5);
36 // GV
37
38 printf("%f", 5);
39 // GV
40
41 printf("%d", 5 + 2);
42 // 7
43
44 printf("5 + 2");
45 // 5 + 2
46
47 printf("%d + %d", 5 + 2);
48 // 7 + GV
49
50 printf("%d * %d = %d", 5, 2, 5+2);
51 // 5 * 2 = 7
52
53 printf("%d * %d = %d", 5, 2, 5*2);
54 // 5 * 2 = 10
55
56 printf("%f", 5/2);
57 // GV
58
59 printf("%d", 5/2);
60 // 2
61
62 printf("%f", 5.0 % 2);
63 // Error
64
65 printf("%d", -5 % -2);
66 // -1
67
68 printf("Hello", "Hai", "Bye");
69 // Hello
70
71 printf("Hello"Hai"Bye");
72 // HelloHaiBye
73
74 printf("Hello");
75 // Error
76
77 printf("Hello");
78 // Hello
79
80 printf("Hello");
81 //Error

```

```

82
83 printf("%d %d %d", "%d %d", 10, 20, 30, 40, 50);
84 // GV 10 20
85
86 printf("10, 20", "%d");
87 // 10, 20
88
89 int ts = 7500;
90 printf("ts");           // ts
91 printf(ts);             // Error
92 printf("Total sal = %d", ts); // Total sal = 7500
93 printf("%d + 1000", ts);   // 7500 + 1000
94 printf("%d", ts + 1000);  // 8500

```

1.6 Variable definition Rules

- Variable definition ways,

```

1 // Method 1:
2 int a;
3 int b;
4 int c;
5
6 // Method 2:
7 int a, b, c;

```

- In the above two style 1st style is recommended because,
 - Subsequent change in variable type in future is easy
 - Writing and updating a comment is easy
- **Variable definition:** Compiler allocates memory for that variable

```

1 int a;
2 int b = 10;

```

- **Variable declaration:** Compiler doesn't allocate memory for that variable

```

1 extern int a;

```

Examples:

```

1 void main()
2 {
3     int a; // Always say datatype to a variable
4     a = 5;
5     printf("%d", a);
6 }
7
8 // Output
9 // 5
10
11 void main()
12 {
13     int i;
14     float f;
15     i = 5;
16     f = 5.5;
17     printf("%d %f", i, f);
18 }
19
20 // Output

```

21 // 5 5.5

- In C programming we can assign any type of data to any type of variable. Internally in computer the compiler will take care.
- In printf function only suitable data is given to format specifier else garbage value is printed.

```
1 void main()
2 {
3     int i;
4     i = 5.5;
5     printf("%d", i); // 5
6 }
7
8 void main()
9 {
10    float f;
11    f = 5;
12    printf("%f", f); // 5.0
13 }
14
15 void main()
16 {
17    int i;
18    i = 5 / 2;
19    printf("%d", i); // 2
20 }
21
22 void main()
23 {
24    float f;
25    f = 5 / 2;
26    printf("%f", f); // 2.0
27 }
28
29 void main()
30 {
31    int i;
32    i = 5.0 / 2;
33    printf("%d", i); // 2
34 }
35
36 void main()
37 {
38    float f;
39    f = 5.0 / 2;
40    printf("%f", f); // 2.5
41 }
```

- More examples

```
1 /***** Valid definition *****/
2
3 int a;
4 a = 5; // Assignment
5
6 int a = 5; // Initialization
7
8 int a = 18 / 3 / 5;
9
```

```

10 int a = 2 + 5;
11
12 int a = 7.65;
13
14 int a, b, c;
15 a = b = c = 10; // Valid
16
17 int a, b, c = a = b = 10;
18
19 int a = a;
20
21 int a1, b1, c1;
22
23 int avgofsalary; // Valid but not readable
24
25 int avg_of_salary;
26
27 int a, A;
28
29 char way2ms;
30
31 char INT;
32
33 int abcdefghijklmnopqrst;
34
35 int _;
36
37 char _1, _2;
38
39 /***** Not a valid *****/
40 int a = b = c = 10;
41
42 int 1a, 1b, 1c;
43
44 int avg of salary; // Not a valid. No white space allowed
45
46 int if;
47
48 int a + b;
49
50 int a, a; // Not a valid. same name not allowed
51
52 float 1606y2;
53
54 int 42shared;
55
56 char `a`, `b`, `c`;

```

- Rules for naming a valid variable Names:
 - An Identifier can contain
 - * a - z
 - * A - Z
 - * 0 - 9
 - * _ (underscore)
 - An Identifier name start with alphabet (or) underscore
 - No whitespace, operator are allowed
 - No keyword, but act as a part of variable
 - * Example: `int intfloat`

- No restriction in length of variable name, but if length is very big then readability is less. Hence programmers recommendation is max of 15 characters

```
1 int pwm_analog_read;
2 int pwm_analog_write;
3 int pwm_digital_read;
4 int pwm_digital_write;
```

1.7 Relational Operators

Operator	Name	Description
<code>==</code>	Equal to	Checks whether two operands are equal
<code>!=</code>	Not equal to	Checks whether two operands are not equal
<code>></code>	Greater than	Checks whether left operand is greater than right operand
<code><</code>	Less than	Checks whether left operand is less than right operand
<code>>=</code>	Greater than or equal to	Checks whether left operand is greater than or equal to right operand
<code><=</code>	Less than or equal to	Checks whether left operand is less than or equal to right operand

```
1 a = 5 + 2; // a is int
2
3 b = 1.5 * 3; // b is float
4
5 c = 4 > 3; // c and d is boolean (but not available in C)
6 d = 9 < 7;
```

- The use of this operator
 - To establish relation between two numbers and perform comparison
 - Result of relation operation is either `true` or `false`
 - `true` and `false` are boolean datatype but C does not support that hence
 - * `true` = Non-zero
 - * `false` = zero

```
1 void main()
2 {
3     int a;
4     a = 4 > 3 > 2;
5     printf("%d", a); // 0
6 }
```

- In Maths,
 - $a > b$ and $b > c$ makes $a > c$. So $a > b > c$
 - $4 > 3$ and $3 > 2$. So $4 > 3 > 2$
- In English, Lets take below two sentences
 - I am watching movie
 - I am eating popcorn
 - Now, Lets combine two statements blindly
 - * I am watching movie eating popcorn (meaning less)
 - So, I am watching movie `and` eating popcorn (meaningful now)
- In C Language
 - $4 > 3 > 2$ makes result in `0` even if we think in mathematically and grammatically
 - So, $(4 > 3) \&\& (3 > 2)$ makes result `1`

```
1 a = 4 + 3 > 2 + 5;
2   = 7 > 2 + 5
3   = 7 > 7
```

```

4   = 0
5
6   a = 6 > 3 + 2 < 8
7   = 6 > 5 < 8
8   = 1 < 8
9   = 1
10
11  a = 6 * 3 >= 5 * 2 + 8 >= 17 + 8 <= 6 * 4 == 8 * 5 != 5 * 3 * 2 + 10 != 40;
12  // Based on the Associativity of operator all are executed from left to right
13  // and last operator is !=. So result always will be 1 or 0

```

1.8 Logical Operator

- If we are combining 2 or more relation operator directly then result is unexpected
- To overcome this C people have introduced logical operators
- Thus use of logical operation is to combine two (or) more relational statement and to get compound statement

Operator	Name	Description
&&	Logical AND	Returns true if both conditions are true
	Logical OR	Returns true if at least one condition is true
!	Logical NOT	Reverses the logical state of operand

A	B	A && B	A B	!A	!B
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	0	1
1	1	1	1	0	0



- Whenever we are having 'N' condition and if we are depend on all 'N' condition at that time we are going for AND logic
- When there are 'N' condition and if we are depend on any 1 condition that time we gone for OR logic
- NOT is used in negative test condition

- In c language every Non-zero is true
- Zero is consider as false
- Whenever we are not giving body for if then by default compiler will consider upto 1st semicolon as if body
- In C only semicolon is valid and it is called null statement
- The purpose of ; is provide delay to next instruction

```

1
2 void main()
3 {
4     int a = 10;
5     int b = 20;
6     int max;
7
8     /* To find max of two integer using if */
9     if(a > b) {

```

```

10     max = a;
11 }
12
13 if(b > a) {
14     max = b
15 }
16
17 /* WAP to find max of 2 integer using only one if */
18 max = b;
19 if(a > b) {
20     max = a;
21 }
22
23 /* WAP to find max of 2 integer using 1 if and without 3rd variable */
24 if(a > b) {
25     b = a;
26 }
27 printf("MAX = %d", b);
28
29 /* WAP to find max of 2 integer without using any if without 3rd variable */
30 max = (a > b) * a + (b < a) * b;
31
32 /* WAP to find max of 2 integer without using any condition */
33 max = (a + b + abs(a - b)) / 2;
34 }

```

Chapter 2

Bitwise Operators

Operator	Name	Description
&	Bitwise AND	Sets bit to 1 if both bits are 1
	Bitwise OR	Sets bit to 1 if at least one bit is 1
^	Bitwise XOR	Sets bit to 1 if bits are different
~	Bitwise NOT / One's Complement	Inverts all bits
«	Left Shift	Shifts bits to left
»	Right Shift	Shifts bits to right

2.1 One's Complement

!	~
Output is 1 or 0	output is integer
!3.14 is 0	~3.14 is error

```
1 void main()
2 {
3     int a;
4     a = ~0;
5     printf("%d", a);
6 }
7
8 /*
9 if int datatype = 2 bytes
10
11 0 = 0000 0000 0000 0000 (result = 0)
12 ~0 = 1111 1111 1111 1111 (result = -1)
13 */
```

- If MSB = 0, then number is +ve and it is original data
- If MSB = 1, then number is -ve and number is in 2's complement.

2.1.1 Binary(2's complement) to decimal

- If MSB = 0, Convert normally from binary to decimal.
- If MSB = 1,
 1. Take 2's complement again
 2. Convert to decimal
 3. Add negative sign

Example

```
1 /*
2 Binary(2's complement) = 1111 1111 1111 0101
3 original data          = 0000 0000 0000 1010
4                               +1
5 -----
6 0000 0000 0000 1011 = 11 (consider this as negative)
7 */
```

```
1 /*
2 ~5
3 ~(0000 0000 0000 0101)
4 1111 1111 1111 1010
5           | |
6           V V
7         4+1 = 5
8           |
9           V
10        5 + 1 = 6 (consider as negative)
11
12 ~5 = -6
13 */
```

```
1 /*
2 ~9
3 ~(0000 0000 0000 1001)
4 1111 1111 1111 0110
5           | |
6           V V
7         8 +1 = 9
8           |
9           V
10        9 + 1 = 10 (consider as negative)
11
12 ~9 = -10
13
14 So,
15 ~0 = -1
16 ~5 = -6
17 ~9 = -10
18 ~37 = -38
19 ~~6 = 5
20 ~~3107 = 3106
21 ~~10906 = 10905
22 ~3.14 = Error (Complement on float is error)
23 */
```

2.1.2 Summary



1. Add 1
2. Change the sign of result

2.2 Left shift Operator

```
1  /*
2  5 << 1 = 0000 0000 0000 0101
3           0000 0000 0000 1010 (shift by 1) = 10
4
5  5 << 2 = 0000 0000 0000 0101
6           0000 0000 0001 0100 (shift by 2) = 20
7
8  -5 << 1 = 1111 1111 1111 1011
9           1111 1111 1111 0110 (shift by 1) = -10
10
11 -6 << 2 = 1111 1111 1111 1010
12           1111 1111 1110 1000 (shift by 2) = -24
13 */
```

2.2.1 Summary



$a \ll b$ is equal to $a * 2^b$

2.3 Right shift Operator

```
1  /*
2  5 >> 1 = 0000 0000 0000 0101
3           0000 0000 0000 0010 = 2
4           |
5           V
6           This position is filled by the value of MSB because signed
7
8  11 >> 2 = 0000 0000 0000 1011
9           0000 0000 0000 0010 = 2
10          ||
11          VV
12          This position is filled by the value of MSB because signed
13
14 -9 >> 1 = 1111 1111 1111 0111
15          1111 1111 1111 1011 = -5
16          |
17          V
18          This position is filled by the value of MSB because signed
19
20 -6 >> 2 = 1111 1111 1111 1010
21          1111 1111 1111 1110 = -2
22          ||
23          VV
24          This position is filled by the value of MSB because signed
25 */
```

2.3.1 Summary

1.

$$5 >> 1 = \frac{5}{2^1} = 2$$

2.

$$11 \gg 2 = \frac{11}{2^2} = 2$$

3.

$$-9 \gg 1 = \sim \left(\frac{\sim -9}{2^1} \right) = \sim \left(\frac{8}{2} \right) = \sim (4) = -5$$

4.

$$-6 \gg 2 = \sim \left(\frac{\sim -6}{2^2} \right) = \sim \left(\frac{5}{4} \right) = \sim (1) = -2$$



$$a \gg b = \begin{cases} \frac{a}{2^b}, & \text{if } a \text{ is positive} \\ \left(\frac{\sim -a}{2^b} \right) & \text{if } a \text{ is negative} \end{cases}$$

- For signed data gap is filled with MSB
- For unsigned data gap is filled with 0
- Shifting like `10 << 40` , `81 >> 100` , `10 << -2` , `8 >> -3` are machine dependant

2.4 Bitwise AND, OR, XOR

AND	XOR	OR
<code>x & 1 => x</code>	<code>x ^ 1 => !X (Toggle)</code>	<code>X 1 => 1 (Set)</code>
<code>x & 0 => 0 (Clear)</code>	<code>X ^ 0 => X</code>	<code>X 0 => X</code>

- `X` is don't care

2.4.1 Summary



1. Set a bit, use OR
2. Clear a bit, use AND
3. Toggle a bit, use XOR

2.5 Problem solving

2.5.1 Check if the nth bit is set

```

1 bool is_bit_set(uint32_t x, uint8_t n)
2 {
3     return (x >> n) & 1U;
4     /* OR: return (x & (1U << n)) != 0; */
5 }

```

2.5.2 Set the nth bit

```

1 uint32_t set_bit(uint32_t x, uint8_t n)
2 {
3     return x | (1U << n);
4 }

```

2.5.3 Clear the nth bit

```
1 uint32_t clear_bit(uint32_t x, uint8_t n)
2 {
3     return x & ~(1U << n);
4 }
```

2.5.4 Toggle the nth bit

```
1 uint32_t toggle_bit(uint32_t x, uint8_t n)
2 {
3     return x ^ (1U << n);
4 }
```

2.5.5 Count number of set bits

```
1 uint8_t count_set_bits(uint32_t x)
2 {
3     uint8_t count = 0;
4     while (x) {
5         x &= x - 1;
6         count++;
7     }
8     return count;
9 }
```

2.5.6 Check if a number is a power of 2

```
1 bool is_power_of_2(uint32_t n)
2 {
3     return (n != 0) && ((n & (n - 1)) == 0);
4 }
```

2.5.7 Swap two numbers using XOR

```
1 void xor_swap(uint32_t *a, uint32_t *b)
2 {
3     if (a != b) {          /* guard against aliasing */
4         *a ^= *b;
5         *b ^= *a;          /* *b = original *a */
6         *a ^= *b;          /* *a = original *b */
7     }
8 }
```

Chapter 3

RTOS

3.1 Fundamentals

3.2 Concepts and Recommendations

3.2.1 The Pitfalls of Dynamic Memory (malloc)

In a standard PC application, you use malloc and free without much thought. In an RTOS, this is often forbidden or highly discouraged.

- **Non-Determinism:** malloc takes a variable amount of time to find a free block. In a real-time system, you need to know exactly how long an operation takes.
- **Fragmentation:** Over time, allocating and freeing small chunks of memory creates “holes.” Eventually, you might have 10KB of free memory total, but not a single continuous 2KB block, causing the system to crash.
- **Solution:** Most RTOS developers use Static Allocation or Memory Pools (fixed-size blocks) to ensure memory is always available and allocation time is constant.

3.2.2 Task States and Timing Analysis

Understanding the State Transition Diagram is crucial. A task isn’t just “running” or “not running.”

- **Ready:** The task is prepared to run but a higher-priority task is currently using the CPU.
- **Running:** The task is currently executing instructions.
- **Blocked/Waiting:** The task is waiting for an event (a timer, a semaphore, or a queue).
- **Suspended:** The task is manually paused and won’t run until another task “resumes” it.

The Interview Question: “What happens if a high-priority task never enters the Blocked state?”

The Answer: It “starves” the system. Lower-priority tasks (including the Idle task which often handles power saving) will never get a single CPU cycle.

3.2.3 Queues: The Right Way to Pass Data

While Semaphores are great for signals, Queues are the gold standard for data.

- **Copy by Value vs. Copy by Reference:** Small data (like a sensor integer) is copied directly into the queue. For large data (like a 1MB image buffer), you put the pointer to the data in the queue.
- **Thread Safety:** Queues are internally protected by the RTOS. You don’t need to wrap a queue in a Mutex; the RTOS handles the locking for you.

3.2.4 Software Timers vs. Hardware Timers

- **Hardware Timers:** Use high-precision peripherals to trigger ISRs. Best for microsecond-level accuracy (e.g., PWM or high-speed sampling).
- **Software Timers:** Managed by the RTOS “Timer Task.” They are easier to use and don’t consume hardware resources, but their accuracy is limited by the RTOS Tick Rate (usually 1ms). If your tick is 1ms, you cannot have a software timer trigger every 0.5ms.

3.2.5 The Tick Hook & System Heartbeat

Every RTOS relies on a hardware timer (usually called the SysTick in ARM Cortex-M) to create a periodic interrupt.

- **The Problem:** You need to perform a small action (like blinking a status LED or updating a watchdog timer) every 1ms, but you don’t want to create a whole new task for it.
- **The Concept:** The Tick Hook is a callback function inside the RTOS kernel that executes every single time the system tick interrupt occurs.
- **The Challenge:** Since this runs inside an ISR context, you must be extremely careful. If the Tick Hook takes too long, the whole system drifts in time.

3.2.6 The Idle Task & Power Management

When no tasks are in the “Ready” state (e.g., everyone is blocked waiting for a sensor or a timer), the RTOS doesn’t just stop. It runs the Idle Task.

- **The Problem:** A battery-powered device needs to last for months. If the CPU is always “Running” (even if just in a loop), the battery will die in days.
- **The Concept:** Tickless Idle. Instead of waking up every 1ms to check the scheduler, the RTOS calculates how long it can safely “sleep” until the next scheduled event and puts the CPU into a Deep Sleep mode.
- **The Interview Question:** “What is the priority of the Idle Task, and why?”
- **The Answer:** It is always Priority 0 (the lowest). If it were higher, it would block every other task from running.

3.2.7 Event Groups (The “Wait for All” logic)

Sometimes, a task needs to wait for multiple things to happen before it can proceed. For example: “Start the Motor only if (Button is Pressed) AND (Temperature is OK) AND (Safety Cover is Closed).”

- **The Problem:** Using three separate semaphores is inefficient and can lead to complex code.
- **The Concept:** Event Groups / Event Flags.

3.3 Problem solving

3.3.1 Odd / Even Print

- **Problem:** Create a two tasks where one task print odd number and another prints even number but while printing the numbers should be in 1, 2, 3, 4, ... and So on.

Code

```
1 SemaphoreHandle_t semOdd = 1; // Start with 1 so Odd Task runs first
2 SemaphoreHandle_t semEven = 0; // Start with 0 so Even Task blocks
3
4 void Task_Odd(void) {
5     int count = 1;
6     while(1) {
7         take(semOdd);           // Wait for turn
8         print(count);
```

```

9         count += 2;
10        give(semEven);           // Signal Even task
11    }
12 }
13
14 void Task_Even(void) {
15     int count = 2;
16     while(1) {
17         take(semEven);           // Wait for turn
18         print(count);
19         count += 2;
20         give(semOdd);            // Signal Odd task
21     }
22 }

```

3.3.2 The Producer-Consumer Pattern

- **The Problem:** Task A reads data from a sensor at a high frequency and places it in a shared circular buffer. Task B processes that data and sends it over UART.
- **The Challenge:** How do you ensure Task B doesn't read an empty buffer and Task A doesn't overwrite data that Task B hasn't processed yet? Which RTOS primitive (Binary Semaphore, Counting Semaphore, or Mutex) is best for signaling Task B that data is ready?
- **Concept:** Use counting semaphores to track empty/full slots and a mutex for data integrity.

Code:

```

1 Semaphore_t semEmpty = BUFFER_SIZE; // Tracks available space
2 Semaphore_t semFull = 0;             // Tracks available data
3 Mutex_t      bufferMutex;           // Protects the buffer indices
4
5 void Producer(void) {
6     while(1) {
7         data = read_sensor();
8
9         take(semEmpty);               // Wait if buffer is full
10        lock(bufferMutex);
11
12        buffer[in] = data;
13        in = (in + 1) % SIZE;
14
15        unlock(bufferMutex);
16        give(semFull);                 // Signal Consumer that data is ready
17    }
18 }
19
20 void Consumer(void) {
21     while(1) {
22         take(semFull);                 // Wait if buffer is empty
23         lock(bufferMutex);
24
25         data = buffer[out];
26         out = (out + 1) % SIZE;
27
28         unlock(bufferMutex);
29         give(semEmpty);               // Signal Producer that space is free
30
31         process(data);

```

```

32     }
33 }

```

3.3.3 Queue Overflow

- **The Problem:** A High-Priority Sensor Task is producing data faster than a Low-Priority Display Task can process and render it. Because the queue has a fixed size, it eventually hits its limit (becomes full).
- **The Challenge:** If the High-Priority task tries to send data to a full queue, it faces a dilemma:
 - **Block:** If it waits for space, the high-priority task stops running, potentially missing critical sensor samples (Timing Jitter).
 - **Discard:** If it skips the data, you lose information.
 - **Overwrite:** If it forces the data in, it might corrupt the message sequence.

The challenge is maintaining system responsiveness without losing the “freshest” data.

- **Concept:** Ping Pong Buffer or Double buffering

Code

```

1  #define BUF_SIZE 100
2
3  // Two physical memory areas
4  float Buffer0[BUF_SIZE];
5  float Buffer1[BUF_SIZE];
6
7  // Signals to tell the Display task which buffer is ready
8  SemaphoreHandle_t semBuffer0Ready;
9  SemaphoreHandle_t semBuffer1Ready;
10 // Signal to tell Sensor task a buffer is empty and safe to use
11 SemaphoreHandle_t semBufferProcessed;
12
13 void Sensor_Task(void) {
14     float *currentBuffer = Buffer0;
15     int index = 0;
16
17     while(1) {
18         currentBuffer[index++] = Read_Sensor();
19
20         if (index >= BUF_SIZE) {
21             // Buffer is full!
22             if (currentBuffer == Buffer0) {
23                 give(semBuffer0Ready); // Tell Display to process Buffer 0
24                 currentBuffer = Buffer1; // Immediately switch to Buffer 1
25             } else {
26                 give(semBuffer1Ready); // Tell Display to process Buffer 1
27                 currentBuffer = Buffer0; // Immediately switch to Buffer 0
28             }
29             index = 0;
30
31             // Wait until the Display task has finished at least ONE of the buffers
32             // This provides the 'Zero Data Loss' protection
33             take(semBufferProcessed);
34         }
35         TaskDelayMs(1);
36     }
37 }
38
39 void Display_Task(void) {

```



```

40 while(1) {
41     // Check if Buffer 0 is ready
42     if (take(semBuffer0Ready, 0) == SUCCESS) {
43         Render_Graph(Buffer0, BUF_SIZE);
44         give(semBufferProcessed); // Signal that Buffer 0 is now free
45     }
46     // Check if Buffer 1 is ready
47     else if (take(semBuffer1Ready, 0) == SUCCESS) {
48         Render_Graph(Buffer1, BUF_SIZE);
49         give(semBufferProcessed); // Signal that Buffer 1 is now free
50     }
51 }
52 }

```

3.3.4 Handling Shared Hardware (The Re-entrancy Problem)

- **The Problem:** Two different tasks (High Priority and Low Priority) both need to write log messages to the same SPI Flash memory.
- **The Challenge:**
 - If the Low Priority task is mid-write and the High Priority task preempts it and tries to write to SPI, the hardware state will crash. How do you protect this shared resource?
 - Why would a Mutex be preferred over a simple Global Boolean flag?
 - * **Atomicity:** Checking a flag and then setting it is a “Read-Modify-Write” operation. In assembly, this is multiple instructions. A task could check the flag, see it’s false, and get preempted before it sets it to true. Another task could then run, see the same false flag, and you now have two tasks accessing the same hardware simultaneously. A Mutex uses hardware-level atomic instructions (like Test-and-Set) to ensure this cannot happen.
 - * **Blocking vs. Polling:** With a boolean flag, a task has to “spin” (loop constantly) to check the flag, wasting CPU cycles. A Mutex puts the task into a Blocked state, allowing other tasks to run. The RTOS scheduler only wakes the task up when the Mutex becomes available.
- **Concept:** Protecting a non-reentrant peripheral using a Mutex. Mutex is preferred over a simple global boolean because priority inversion is handled by mutex using priority inheritance.

Code:

```

1  Mutex_t spiMutex;
2
3  void Log_Data(char* msg) {
4      lock(spiMutex);      // Only one task can own the SPI bus
5
6      SPI_Select_Chip();
7      SPI_Write(msg);
8      SPI_Deselect_Chip();
9
10     unlock(spiMutex);    // Release for other tasks
11 }
12
13 void Task_HighPriority(void) {
14     Log_Data("High Priority Alert!");
15 }
16
17 void Task_LowPriority(void) {
18     Log_Data("Routine system log.");
19 }

```

3.3.5 The Deadlock Scenario

- **The Problem:** Task 1 needs Resource A then Resource B. Task 2 needs Resource B then Resource A.
- **The Challenge:** Describe the “Deadly Embrace” where both tasks end up waiting forever. What are two architectural ways to prevent this in a real-time system?
 - **Resource Hierarchy (Lock Ordering):** Assign a numerical “rank” to every resource (e.g., SPI=1, I2C=2, UART=3). Rule: A task can only acquire a resource with a higher rank than the ones it already holds. This makes it mathematically impossible to form a circular wait.
 - **Timeout-Based Acquisition:** Instead of waiting forever (wait_forever), use a timeout (e.g., take_mutex(lock, 100ms)). If the task can’t get all the resources it needs within the time limit, it must release all its held locks and try again later. This breaks the “Hold and Wait” condition of deadlocks.
- **Concept:** To prevent circular wait, tasks must acquire resources in the exact same numerical order.

Code:

```
1  Mutex_t Mutex_A; // Resource 1
2  Mutex_t Mutex_B; // Resource 2
3
4  void Task_Safe(void) {
5      while(1) {
6          // ALWAYS take A before B
7          lock(Mutex_A);
8          lock(Mutex_B);
9
10         // Access shared resources
11
12         unlock(Mutex_B);
13         unlock(Mutex_A);
14     }
15 }
```

3.3.6 Priority Inversion:

- **The Problem:** A High-priority task is waiting for a Mutex held by a Low-priority task. A Medium-priority task starts running, preventing the Low-priority task from finishing and releasing the Mutex.
- **The Challenge:** How does the “Priority Inheritance” protocol solve this? Can you explain the difference between Priority Inheritance and Priority Ceiling?
 - **Priority Inheritance:** The priority of the lower-priority task holding the lock is boosted only when a higher-priority task tries to acquire that same lock. It is “reactive.”
 - **Priority Ceiling:** Every resource (Mutex) is assigned a “Ceiling” priority (the priority of the highest task that could ever use it). As soon as any task grabs that Mutex, its priority is immediately boosted to that Ceiling, regardless of whether a higher-priority task is actually waiting. This is “proactive” and effectively prevents deadlocks as well.
- **Concept:** Illustrating how a Mutex with inheritance automatically fixes the “Unbounded” inversion.

Code

```
1  /*
2   RTOS Settings: Mutex must be created with Priority Inheritance ENABLED.
3   Low Priority (LP) holds Mutex.
4   High Priority (HP) wants Mutex.
5   LP is boosted to HP's priority until it unlocks.
6  */
7
8  void Task_Low(void) {
9      lock(mutexInherit);
```

```

10 // Even if Task_Medium tries to preempt here,
11 // it fails because Task_Low is now "High Priority"
12 unlock(mutexInherit); // Priority returns to Low
13 }

```

3.3.7 Interrupt to Task Communication

- **The Problem:** An external GPIO interrupt (ISR) triggers every time a button is pressed. You want to toggle an LED in a Task.
- **The Challenge:** Why shouldn't you toggle the LED or perform heavy logic directly inside the ISR? Explain the concept of "Deferred Interrupt Processing" using a semaphore or task notification.
 - Why avoid heavy logic/LED toggling in ISRs?
 1. **Jitter:** While an ISR is running, other interrupts of the same or lower priority are blocked. This creates "latency" for other parts of the system.
 2. **Stack Overflow:** ISRs often use a limited system stack. Complex logic or printf calls can easily crash the system.
 3. **Non-deterministic behavior:** Many RTOS functions (like delay or lock_mutex) can block. You cannot block inside an ISR, as there is no "task context" to switch away from.
 - **Deferred Interrupt Processing:** Instead of doing the work in the ISR, the ISR simply "gives" a semaphore or a task notification and exits. The RTOS then sees a high-priority "Worker Task" is now ready to run. The worker task wakes up, does the heavy lifting (like writing to an SD card or toggling an LED), and then goes back to sleep. This ensures the hardware interrupt is cleared in microseconds, keeping the system responsive.

Concept: Keep the ISR short by offloading logic to a task using a binary semaphore.

Code

```

1 Semaphore_t semButton;
2
3 // ISR (Interrupt Service Routine)
4 void GPIO_ISR(void) {
5     // Keep this extremely fast!
6     give_from_ISR(semButton); // Wake up the task
7 }
8
9 // Processing Task
10 void Task_ButtonHandler(void) {
11     while(1) {
12         take(semButton); // Blocks here until button is pressed
13         // Perform heavy work (e.g., Debouncing, Network requests)
14         Toggle_LED();
15         printf("Button Pressed!\n");
16     }
17 }

```